

LAPORAN PRAKTIKUM PEKAN 9



MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 7 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar *Binary Tree* dan *Graph*.

Praktikum ini bertujuan untuk memahami konsep, struktur, serta implementasi *Binary Tree* dan *Graph* dalam pemrograman. Melalui praktikum ini, mahasiswa dapat mempelajari cara membangun, mengelola, dan melakukan penelusuran data pada kedua struktur data tersebut sehingga dapat digunakan untuk menyelesaikan berbagai permasalahan komputasi secara lebih efisien.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 09 Juni 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB 1	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB 2	2
PEMBAHASAN	2
2.1 <i>Binary Tree</i> dan <i>Graph</i>	2
2.2 Class Node.....	3
2.3 Class BTree	6
2.4 Class BTreeDriver.....	9
2.5 Class GraphTraversal.....	11
2.6 Tugas Praktikum Pekan 9	15
BAB 3	41
KESIMPULAN.....	41
DAFTAR PUSTAKA	42
LAMPIRAN.....	43

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu komponen penting dalam ilmu komputer yang digunakan untuk mengatur dan mengelola data agar dapat diproses secara efektif dan efisien. Pemilihan struktur data yang tepat sangat memengaruhi terhadap kinerja suatu program, terutama ketika menangani data dalam jumlah besar atau memiliki hubungan yang kompleks.

Binary Tree merupakan struktur data non-linear yang tersusun atas simpul (*node*) yang saling terhubung dalam bentuk hierarki. Setiap simpul pada *Binary Tree* dapat memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Struktur ini banyak digunakan dalam proses pencarian, pengurutan data, serta implementasi berbagai algoritma yang membutuhkan pengelolaan data secara hierarkis.

Selain *Binary Tree*, *Graph* juga merupakan struktur data non-linear yang digunakan untuk merepresentasikan hubungan antar objek. *Graph* terdiri atas kumpulan simpul (*vertex*) dan sisi (*edge*) yang menghubungkan simpul-simpul tersebut. Struktur data ini banyak diterapkan dalam berbagai bidang, seperti sistem navigasi, jaringan komputer, media sosial, dan analisis hubungan antar data.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar *Binary Tree* dan *Graph*.
2. Mengetahui cara merepresentasikan data menggunakan *Binary Tree* dan *Graph*.
3. Mengimplementasikan *Binary Tree* dan *Graph* ke dalam program Java.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai *Binary Tree* dan *Graph*.
2. Meningkatkan kemampuan pemrograman dalam bahasa Java.
3. Melatih keterampilan dalam mengelola kode program secara terstruktur.

BAB 2

PEMBAHASAN

2.1 *Binary Tree* dan *Graph*

Binary Tree merupakan salah satu struktur data non-linear yang menyimpan data dalam bentuk hierarki. Struktur ini terdiri dari simpul (*node*) yang saling terhubung, di mana setiap simpul dapat memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Simpul paling atas disebut *root*, sedangkan simpul yang tidak memiliki anak disebut *leaf node*. *Binary Tree* banyak digunakan dalam pengolahan data karena mampu mempercepat proses pencarian, penyisipan, dan penghapusan data dibandingkan struktur data linear pada kondisi tertentu. Selain itu, *Binary Tree* memiliki beberapa metode penelusuran (*traversal*) yang umum digunakan, yaitu *preorder*, *inorder*, dan *postorder traversal*. Masing-masing metode memiliki urutan kunjungan simpul yang berbeda sesuai dengan kebutuhan pengolahan data.

Dalam perkembangannya, *Binary Tree* memiliki berbagai variasi, salah satunya adalah *Binary Tree Search* (BST). Pada BST, nilai yang lebih kecil dari *root* ditempatkan di *subtree* kiri, sedangkan nilai yang lebih besar ditempatkan di *subtree* kanan. Aturan ini memungkinkan proses pencarian data dilakukan secara lebih efisien karena tidak perlu memeriksa seluruh simpul yang ada. Struktur *Binary Tree* banyak diterapkan dalam sistem basis data, kompilator, sistem file, dan berbagai aplikasi yang membutuhkan pengelolaan data secara terstruktur dan cepat.

Selain *Binary Tree* *Graph* juga merupakan struktur data non-linear yang digunakan untuk merepresentasikan hubungan antar objek. *Graph* terdiri atas sekumpulan simpul yang disebut *vertex* dan hubungan antar simpul yang disebut *edge*. Berbeda dengan *Binary Tree* yang memiliki struktur hierarkis dan satu *root* utama, *Graph* memiliki bentuk yang lebih fleksibel karena setiap *vertex* dapat terhubung dengan banyak *vertex* lainnya. Berdasarkan arah hubungan antar *vertex*, *Graph* dibedakan menjadi *Directed Graph* (graf berarah) dan *Undirected Graph* (graf tidak berarah). Selain itu, *Graph* juga dapat berupa *Weighted Graph* apabila setiap *edge* memiliki nilai bobot tertentu.

Graph banyak digunakan untuk memodelkan berbagai permasalahan dunia nyata yang melibatkan hubungan antar entitas. Contohnya adalah jaringan

komputer, peta navigasi, media sosial, jaringan transportasi, serta sistem rekomendasi. Untuk melakukan penelusuran pada *Graph*, terdapat dua algoritma yang umum digunakan, yaitu *Breadth First Search* (BFS) dan *Depth First Search* (DFS). BFS melakukan penelusuran secara melebar dengan mengunjungi seluruh tetangga suatu *vertex* terlebih dahulu, sedangkan DFS melakukan penelusuran secara mendalam hingga mencapai *vertex* terakhir sebelum kembali ke *vertex* sebelumnya. Kedua algoritma tersebut memiliki peran penting dalam pencarian jalur, analisis jaringan, serta penyelesaian berbagai masalah komputasi yang melibatkan hubungan antar data.

2.2 Class Node

Berikut kode program Java mengenai class Node :

```
public class Node_2511532027 {
    int data_2027;
    Node_2511532027 left_2027;
    Node_2511532027 right_2027;
    public Node_2511532027(int data_2027) {
        this.data_2027 = data_2027;
        left_2027 = null;
        right_2027 = null;
    }
    public void setLeft_2027(Node_2511532027 node_2027) {
        if (left_2027 == null)
            left_2027 = node_2027;
    }
    public void setRight_2027(Node_2511532027 node_2027) {
        if (right_2027 == null)
            right_2027 = node_2027;
    }
    public Node_2511532027 getLeft_2027() {
        return left_2027;
    }
    public Node_2511532027 getRight_2027() {
        return right_2027;
    }
    public int getData_2027() {
        return data_2027;
    }
}
```

```

    }
    public void setData_2027(int data_2027) {
        this.data_2027 = data_2027;
    }
    void printPreorder_2027(Node_2511532027 node_2027) {
        if (node_2027 == null)
            return;
        System.out.print(node_2027.data_2027 + " ");
        printPreorder_2027(node_2027.left_2027);
        printPreorder_2027(node_2027.right_2027);
    }
    void printPostorder_2027(Node_2511532027 node_2027) {
        if (node_2027 == null)
            return;
        printPostorder_2027(node_2027.left_2027);
        printPostorder_2027(node_2027.right_2027);
        System.out.print(node_2027.data_2027 + "");
    }
    void printInorder_2027(Node_2511532027 node_2027) {
        if (node_2027 == null)
            return;
        printInorder_2027(node_2027.left_2027);
        System.out.print(node_2027.data_2027 + "");
        printInorder_2027(node_2027.right_2027);
    }
    public String print_2027() {
        return this.print_2027("", true, "");
    }
    public String print_2027(String prefix_2027, boolean isTail_2027,
String sb_2027) {
        if (right_2027 != null) {
            right_2027.print_2027(prefix_2027 + (isTail_2027 ?
"| " : " "), false, sb_2027);
        }
        System.out.println(prefix_2027 + (isTail_2027 ? "\\-- " :
"/-- ") + data_2027);
        if (left_2027 != null) {

```

```

        left_2027.print_2027(prefix_2027 + (isTail_2027 ? "
" : "| "), true, sb_2027);
    }
    return sb_2027;
}
}

```

Kode program tersebut digunakan untuk merepresentasikan sebuah simpul (*node*) pada struktur data *Binary Tree*. Kelas `Node_2511532027` memiliki atribut `data_2027` yang berfungsi untuk menyimpan nilai data pada simpul, serta atribut `left_2027` dan `right_2027` yang digunakan untuk menyimpan referensi ke anak kiri dan anak kanan dari simpul tersebut. Konstruktor `Node_2511532027(int data_2027)` digunakan untuk menginisialisasi nilai data pada simpul dan mengatur anak kiri maupun kanan menjadi null, yang menandakan bahwa simpul belum memiliki cabang.

Kode program tersebut juga digunakan untuk mengatur dan mengakses data maupun hubungan antar simpul dalam *Binary Tree*. Method `setLeft_2027()` dan `setRight_2027()` berfungsi untuk menambahkan anak kiri dan anak kanan pada simpul, sedangkan method `getLeft_2027()`, `getRight_2027()`, dan `getData_2027()` digunakan untuk mengambil informasi yang tersimpan pada simpul. Selain itu, method `setData_2027()` digunakan untuk mengubah nilai data yang terdapat pada simpul apabila diperlukan.

Kode program tersebut juga digunakan untuk melakukan penelusuran (*traversal*) pada *Binary Tree* menggunakan metode rekursif. Method `printPreorder_2027()` melakukan traversal dengan urutan akar, anak kiri, kemudian anak kanan. Method `printInorder_2027()` melakukan traversal dengan urutan anak kiri, akar, kemudian anak kanan. Sementara itu, method `printPostorder_2027()` melakukan traversal dengan urutan anak kiri, anak kanan, kemudian akar. Ketiga method tersebut memungkinkan seluruh simpul pada pohon dikunjungi dan ditampilkan sesuai dengan urutan traversal yang dipilih.

Selain itu, kode program tersebut digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk visual menyerupai diagram pohon. Method `print_2027()` dan `print_2027(String prefix_2027, boolean isTail_2027, String sb_2027)` bekerja

secara rekursif untuk mencetak setiap simpul serta hubungan antar *parent* dan *child* menggunakan karakter khusus seperti \-- dan --/.

2.3 Class BTree

Berikut kode program Java mengenai class BTree :

```
public class BTree_2511532027 {
    private Node_2511532027 root_2027;
    private Node_2511532027 currentNode_2027;
    public BTree_2511532027() {
        root_2027 = null;
    }
    public boolean search_2027(int data_2027) {
        return search_2027(root_2027, data_2027);
    }
    private boolean search_2027(Node_2511532027 node_2027, int
data_2027) {
        if (node_2027.getData_2027() == data_2027)
            return true;
        if (node_2027.getLeft_2027() != null)
            if (search_2027(node_2027.getLeft_2027(),
data_2027))
                return true;
        if (node_2027.getRight_2027() != null)
            if (search_2027(node_2027.getRight_2027(),
data_2027))
                return true;
        return false;
    }
    public void printInorder_2027() {
        root_2027.printInorder_2027(root_2027);
    }
    public void printPreorder_2027() {
        root_2027.printPreorder_2027(root_2027);
    }
    public void printPostorder_2027() {
        root_2027.printPostorder_2027(root_2027);
    }
    public Node_2511532027 getRoot_2027() {
```

```

        return root_2027;
    }
    public boolean isEmpty_2027() {
        return root_2027 == null;
    }
    public int countNodes_2027() {
        return countNodes_2027(root_2027);
    }
    private int countNodes_2027(Node_2511532027 node_2027) {
        int count_2027 = 1;
        if (node_2027 == null) {
            return 0;
        } else {
            count_2027 +=
countNodes_2027(node_2027.getLeft_2027());
            count_2027 +=
countNodes_2027(node_2027.getRight_2027());
            return count_2027;
        }
    }
    public void print_2027() {
        root_2027.print_2027();
    }
    public Node_2511532027 getCurrent_2027() {
        return currentNode_2027;
    }
    public void setCurrent_2027(Node_2511532027 node_2027) {
        this.currentNode_2027 = node_2027;
    }
    public void setRoot_2027(Node_2511532027 node_2027) {
        this.root_2027 = node_2027;
    }
}

```

Kode program tersebut digunakan untuk mengelola struktur data *Binary Tree* melalui kelas BTree_2511532027. Kelas ini memiliki atribut root_2027 yang berfungsi sebagai akar dari pohon biner serta atribut currentNode_2027 yang digunakan untuk menyimpan simpul yang sedang aktif atau sedang proses.

Konstruktor BTree_2511532027() digunakan untuk menginisialisasi pohon dalam keadaan kosong dengan nilai root_2027 bernilai null.

Kode program tersebut juga digunakan untuk melakukan proses pencarian data pada *Binary Tree*. Method search_2027(int data_2027) memanggil method rekursif search_2027(Node_2511532027 node_2027, int data_2027) yang bertugas menelusuri seluruh simpul dalam pohon. Proses pencarian dimulai dari akar, kemudian dilanjutkan ke anak kiri dan anak kanan hingga data yang dicari ditemukan. Jika data ditemukan maka method akan mengembalikan nilai true, sedangkan jika seluruh simpul telah diperiksa dan data tidak ditemukan maka method akan mengembalikan nilai false.

Selain itu, kode program tersebut digunakan untuk melakukan penelusuran pada *Binary Tree*. Method printInorder_2027(), printPreorder_2027(), dan printPostorder_2027() memanfaatkan method traversal yang terdapat pada kelas Node_2511532027 untuk menampilkan isi pohon sesuai dengan urutan traversal yang dipilih. Traversal ini berguna untuk melihat isi dan susunan data yang tersimpan di dalam *Binary Tree* berdasarkan metode penelusuran tertentu.

Kode program tersebut juga digunakan untuk memeriksa kondisi dan menghitung jumlah simpul yang terdapat pada pohon. Method isEmpty_2027() berfungsi untuk mengetahui apakah *Binary Tree* masih kosong atau sudah memiliki simpul. Sementara itu, method countNodes_2027() dan method rekursif countNodes_2027(Node_2511532027 node_2027) digunakan untuk menghitung jumlah seluruh simpul yang ada pada *Binary Tree*. Perhitungan dilakukan dengan mengunjungi setiap simpul pada cabang kiri dan kanan hingga seluruh simpul berhasil dihitung.

Selanjutnya, kodeprogram tersebut digunakan untuk mengatur serta mengakses informasi penting pada *Binary Tree*. Method getRoot_2027() dan setRoot_2027() digunakan untuk mengambil maupun mengubah akar pohon. Method getCurrent_2027() dan setCurrent_2027() digunakan untuk mengelola simpul yang sedang aktif. Selain itu, method print_2027() digunakan untuk menampilkan struktur *Binary Tree* secara visual sehingga hubungan antara akar, anak kiri, dan anak kanan dapat terlihat dengan lebih jelas. Kelas ini berfungsi sebagai pengelola utama yang menangani operasi-operasi dasar pada *Binary Tree*.

2.4 Class BTreeDriver

Berikut kode program Java mengenai class BTreeDriver :

```
public class BTreeDriver_2511532027 {
    public static void main (String[] args) {
        // Membuat Pohon
        BTree_2511532027 tree_2027 = new BTree_2511532027();
        System.out.print("Jumlah simpul awal pohon: ");
        System.out.println(tree_2027.countNodes_2027());
        // Menambahkan simpul data 1
        Node_2511532027 root_2027 = new Node_2511532027(1);
        // Menjadikan simpul 1 sebagai root
        tree_2027.setRoot_2027(root_2027);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_2027.countNodes_2027());
        Node_2511532027 node2_2027 = new Node_2511532027(2);
        Node_2511532027 node3_2027 = new Node_2511532027(3);
        Node_2511532027 node4_2027 = new Node_2511532027(4);
        Node_2511532027 node5_2027 = new Node_2511532027(5);
        Node_2511532027 node6_2027 = new Node_2511532027(6);
        Node_2511532027 node7_2027 = new Node_2511532027(7);
        Node_2511532027 node8_2027 = new Node_2511532027(8);
        Node_2511532027 node9_2027 = new Node_2511532027(9);
        root_2027.setLeft_2027(node2_2027);
        node2_2027.setLeft_2027(node4_2027);
        node2_2027.setRight_2027(node5_2027);
        node4_2027.setRight_2027(node8_2027);
        root_2027.setRight_2027(node3_2027);
        node3_2027.setLeft_2027(node6_2027);
        node3_2027.setRight_2027(node7_2027);
        node6_2027.setLeft_2027(node9_2027);

        // Set root
        tree_2027.setCurrent_2027(tree_2027.getRoot_2027());
        System.out.println("Menampilkan simpul terakhir: ");

        System.out.println(tree_2027.getCurrent_2027().getData_2027());
        System.out.println("Jumlah simpul; setelah simpul 7
        ditambahkan");
    }
}
```

```

        System.out.println(tree_2027.countNodes_2027());
        System.out.println("InOrder: ");
        tree_2027.printPreorder_2027();
        System.out.println("\nPostorder : ");
        tree_2027.printPostorder_2027();
        System.out.println("\nMenampilkan simpul dalam bentuk
pohon");
        tree_2027.print_2027();
    }
}

```

Kode program tersebut digunakan untuk menguji implementasi struktur data *Binary Tree* yang telah dibuat pada kelas `Node_2511532027` dan `BTree_2511532027`. Program diawali dengan pembuatan objek `tree_2027` dari kelas `BTree_2511532027` yang merepresentasikan sebuah pohon biner. Selanjutnya, program menampilkan jumlah simpul awal pada pohon menggunakan method `countNodes_2027()`. Karena pohon baru dibuat dan belum memiliki simpul, maka jumlah simpul yang ditampilkan adalah nol.

Kode program tersebut juga digunakan untuk membangun struktur *Binary Tree* secara manual dengan membuat beberapa objek simpul menggunakan kelas `Node_2511532027`. Simpul bernilai 1 dijadikan sebagai akar melalui method `setRoot_2027()`. Setelah itu, simpul bernilai 2 hingga 9 dibuat dan dihubungkan satu sama lain menggunakan method `setLeft_2027()` dan `setRight_2027()`. Proses ini membentuk hubungan *parent* dan *child* sehingga terbentuk sebuah pohon biner dengan struktur tertentu. Setelah akar ditambahkan, program kembali menghitung jumlah simpul untuk memastikan bahwa pohon telah berisi data.

Selanjutnya, kode program tersebut digunakan untuk mengatur simpul yang sedang aktif menggunakan method `setCurrent_2027()`. Simpul aktif diatur menjadi akar pohon dengan mengambil nilai dari method `getRoot_2027()`. Kemudian, program menampilkan data yang terdapat pada simpul aktif menggunakan method `getCurrent_2027().getData_2027()`. Langkah ini menunjukkan bagaimana sebuah simpul dapat diakses dan dikelola melalui atribut `currentNode_2027` yang terdapat pada kelas *Binary Tree*.

Selain itu, kode program tersebut digunakan untuk menghitung dan menampilkan jumlah seluruh simpul yang telah berhasil ditambahkan ke dalam

Binary Tree. Perhitungan dilakukan menggunakan method `countNodes_2027()` yang menelusuri seluruh cabang pohon secara rekursif. Dengan adanya proses ini, pengguna dapat mengetahui jumlah data yang tersimpan pada struktur *Binary Tree* yang telah dibangun.

Kode program tersebut juga digunakan untuk menampilkan isi pohon menggunakan metode traversal. Method `printPreorder_2027()` digunakan untuk menampilkan data dengan urutan akar, anak kiri, dan anak kanan, sedangkan method `printPostorder_2027()` digunakan untuk menampilkan data dengan urutan anak kiri, anak kanan, kemudian akar. Traversal ini berguna untuk memahami cara data ditelusuri dan diproses dalam *Binart Tree*.

Terakhir, kode program tersebut digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk visual menyerupai diagram pohon melalui method `print_2027()`. Tampilan ini memperlihatkan hubungan antar akar, anak kiri, dan anak kanan secara lebih jelas sehingga memudahkan pengguna dalam memahami bentuk dan susunan simpul yang terdapat pada *Binary Tree*.

2.5 Class GraphTraversal

Berikut kode program Java mengenai class `GraphTraversal` :

```
import java.util.*;

public class GraphTraversal_2511532027 {
    private Map<String, List<String>> graph_2027 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_2027(String node1_2027, String node2_2027) {
        graph_2027.putIfAbsent(node1_2027, new ArrayList<>());
        graph_2027.putIfAbsent(node2_2027, new ArrayList<>());
        graph_2027.get(node1_2027).add(node2_2027);
        graph_2027.get(node2_2027).add(node1_2027);
    }

    // Menampilkan graf awal
    public void printGraph_2027() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_2027 : graph_2027.keySet()) {
            System.out.print(node_2027 + " -> ");
        }
    }
}
```

```

        List<String> neighbors_2027 =
graph_2027.get(node_2027);
        System.out.println(String.join(" ",
neighbors_2027));
    }
    System.out.println();
}
// DFS rekursif
public void dfs_2027(String start_2027) {
    Set<String> visited_2027 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_2027(start_2027, visited_2027);
    System.out.println();
}
private void dfsHelper_2027(String current_2027,
Set<String> visited_2027) {
    if (visited_2027.contains(current_2027)) return;
    visited_2027.add(current_2027);
    System.out.print(current_2027 + " ");
    for (String neighbor_2027 :
graph_2027.getDefault(current_2027, new ArrayList<>())) {
        dfsHelper_2027(neighbor_2027, visited_2027);
    }
}
// BFS iteratif
public void bfs_2027(String start_2027) {
    Set<String> visited_2027 = new HashSet<>();
    Queue<String> queue_2027 = new LinkedList<>();
    queue_2027.add(start_2027);
    visited_2027.add(start_2027);
    System.out.println("Penelusuran BFS:");
    while (!queue_2027.isEmpty()) {
        String current_2027 = queue_2027.poll();
        System.out.print(current_2027 + " ");
        for (String neighbor_2027 :
graph_2027.getDefault(current_2027, new ArrayList<>())) {
            if
(!visited_2027.contains(neighbor_2027)) {

```

```

        queue_2027.add(neighbor_2027);
        visited_2027.add(neighbor_2027);
    }
}

}

System.out.println();
}

// Main
public static void main (String[] args) {
    GraphTraversal_2511532027 graph_2027 = new
GraphTraversal_2511532027();

    // Contoh graf : A-B, A-C, B-D, B-E
    graph_2027.addEdge_2027("A", "B");
    graph_2027.addEdge_2027("A", "C");
    graph_2027.addEdge_2027("B", "D");
    graph_2027.addEdge_2027("B", "E");
    // Cetak graf awal
    System.out.println("Graf Awal adalah: ");
    graph_2027.printGraph_2027();
    // Lakukan penelusuran
    graph_2027.dfs_2027("A");
    graph_2027.bfs_2027("A");

}
}

```

Kode program tersebut digunakan untuk mengimplementasikan struktur data *Graph* serta melakukan proses penelusuran menggunakan algoritma DFS dan BFS. Kelas `GraphTraversal_2511532027` menyimpan representasi graf menggunakan struktur data `Map<String, List<String>>` yang berfungsi sebagai *adjacency list*, yaitu daftar yang menyimpan hubungan antara setiap simpul dengan simpul-simpul yang bertetangga. Penggunaan daftar ini memungkinkan penyimpanan graf menjadi lebih efisien dan mudah untuk ditelusuri.

Kode program tersebut juga digunakan untuk menambahkan hubungan atau *edge* antar simpul melalui method `addEdge_2027()`. Method ini terlebih dahulu memastikan bahwa kedua simpul sudah tersedia dalam graf menggunakan

putIfAbsent()). Setelah itu, simpul tujuan ditambahkan ke daftar tetangga masing-masing simpul. Karena graf yang digunakan merupakan graf tak berarah, maka hubungan ditambahkan pada kedua arah, yaitu dari simpul pertama ke simpul kedua dan dari simpul kedua ke simpul pertama.

Selain itu, kode program tersebut digunakan untuk menampilkan struktur graf yang telah dibuat menggunakan method `printGraph_2027()`. Method ini menampilkan setiap simpul beserta daftar tetangganya dalam bentuk *adjacency list*. Proses ini memudahkan pengguna untuk melihat hubungan antar simpul yang terdapat pada graf sebelum dilakukan proses penelusuran.

Kode program tersebut juga digunakan untuk melakukan penelusuran graf menggunakan algoritma DFS melalui method `dfs_2027()`. Algoritma DFS bekerja dengan mengunjungi suatu simpul terlebih dahulu, kemudian menelusuri simpul-simpul tetangganya hingga mencapai simpul terdalam sebelum kembali ke simpul sebelumnya. Implementasi DFS dilakukan secara rekursif menggunakan method `dfsHelper_2027()`. Untuk mencegah kunjungan berulang pada simpul yang sama, digunakan struktur data `HashSet` yang berfungsi menyimpan daftar simpul yang telah dikunjungi.

Selanjutnya, kode program tersebut digunakan untuk melakukan penelusuran graf menggunakan algoritma BFS melalui method `bfs_2027()`. Berbeda dengan DFS, algoritma BFS menelusuri graf berdasarkan tingkat atau level. Penelusuran dimulai dari simpul awal, kemudian mengunjungi seluruh tetangga yang berada pada level yang sama sebelum berpindah ke level berikutnya. Implementasi BFS memanfaatkan struktur data *Queue* untuk mengatur urutan kunjungan simpul dan `HashSet` untuk mencatat simpul yang telah dikunjungi sehingga tidak terjadi pengulangan penelusuran.

Pada method `main()`, kode program tersebut digunakan untuk membuat sebuah graf sederhana yang terdiri dari lima simpul, yaitu A, B, C, D, dan E. Hubungan antar simpul dibentuk menggunakan method `addEdge_2027()`, yaitu A-B, A-C, B-D, dan B-E. Setelah graf berhasil dibuat, program menampilkan struktur graf dalam bentuk *adjacency list* menggunakan method `printGraph_2027()`. Selanjutnya, program melakukan penelusuran menggunakan algoritma DFS dan BFS yang dimulai dari simpul A.

2.6 Tugas Praktikum Pekan 9

Berikut tugas praktikum yang diberikan :

Tugas BFS dan DFS

Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (*Breadth First Search*) dan DFS (*Depth First Search*) menggunakan bahasa Java dengan antarmuka GUI.

Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph.

Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa:

- Gedung perkuliahan
- Fasilitas
- Universitas
- Sekolah
- Rumah sakit
- Hotel
- Tempat wisata
- Bandara
- Area lokasi lainnya

Program harus mampu:

- Menampilkan graph dalam bentuk visual
- Menentukan titik awal (Start)
- Menentukan titik tujuan (Goal)
- Melakukan pencarian menggunakan BFS
- Melakukan pencarian menggunakan DFS
- Menampilkan jalur yang ditemukan

Gambar 2.1

- Menampilkan urutan simpul yang dikunjungi
- Urutan node yang dikunjungi
- Jumlah node yang dieksplorasi

Aturan Penamaan

1. Penamaan class dan file

Gunakanakhiran `.h` untuk kelas pada nama class dan nama file.

Contoh:

ex : `PetaKampus_2511532006`

2. Penamaan Variabel

Gunakan 4 digit terakhir pada setiap nama variabel

ex : `graph_2019`

Struktur Tugas

Program utama memiliki:

a. Vertex and Edge

- Minimal 10 simpul (vertex)
- Minimal 15 sisi (edge)
- Nama simpul harus merepresentasikan lokasi pada peta yang dibuat

b. Method yang Wajib Ada

1. `BFS()`

Melakukan pencarian menggunakan algoritma Breadth First Search.

2. `DFS()`

Melakukan pencarian menggunakan algoritma Depth First Search.

3. `displayGraph()`

Menampilkan graph pada GUI.

4. `displayPath()`

Gambar 2.2

Menampilkan jalur yang ditemukan.

5. `resetGraph()`

Mengembalikan kondisi graph ke keadaan awal.

c. GUI Program

GUI minimal memiliki

1. Area visualisasi graph
2. ComboBox atau TextField untuk memilih lokasi awal
3. ComboBox atau TextField untuk memilih lokasi tujuan
4. Tombol BFS
5. Tombol DFS
6. Tombol Reset
7. Area hasil pencarian
8. Berikan warna berbeda pada node yang telah dikunjungi.
9. Menampilkan jumlah node yang dikunjungi.

d. Ketentuan Graph

1. Setiap mahasiswa wajib membuat graph yang diberikan.
2. Tidak diperbolehkan menggunakan graph yang sama persis dengan mahasiswa lain.
3. Graph harus memiliki minimal 10 node dan 15 edge.
4. Mahasiswa bebas menentukan bentuk dan jenis peta.
5. Mahasiswa wajib menjelaskan struktur graph, cara kerja BFS dan DFS yang dibuat pada laporan.

Output yang Dikumpulkan

1. Source code Java.
2. Screenshot program saat dijalankan.
3. Laporan singkat (PDF) yang baik.

Gambar 2.3

- Deskripsi peta yang dibuat.
- Jumlah node dan edge.
- Hasil BFS.
- Hasil DFS.
- Kesimpulan perbandingan BFS dan DFS.

tampilan



Gambar 2.4

Berikut kode program Java dari tugas praktikum :

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import javax.swing.border.EmptyBorder;
import java.awt.BasicStroke;
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
```

```

import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.Point;
import java.awt.Rectangle;
import java.awt.RenderingHints;
import java.awt.Color;
import java.awt.Component;
import java.awt.Cursor;
import java.awt.Font;
import java.awt.FontMetrics;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;
import javax.swing.plaf.basic.BasicComboBoxRenderer;

public class PetaKota_2511532027 extends JFrame {

    /**
     *
     */
    private static final long serialVersionUID = 1L;
    private Map<String, List<String>> graph_2027;
    private Map<String, Point> nodePos_2027;
    private List<String> visitedNodes_2027;
    private Set<String> highlightPath_2027;
    private Set<String> highlightVisited_2027;

```

```

private JComboBox<String> cbStart_2027;
private JComboBox<String> cbGoal_2027;
private JTextArea      resultArea_2027;
private GraphPanel_2027 graphPanel_2027;

private static final Color C_BG      = new Color(15, 23, 42);
private static final Color C_PANEL   = new Color(30, 41, 59);
private static final Color C_CARD    = new Color(51, 65, 85);
private static final Color C_ACCENT  = new Color(99, 102, 241);
private static final Color C_GREEN   = new Color(16, 185, 129);
private static final Color C_YELLOW  = new Color(245, 158, 11);
private static final Color C_RED     = new Color(239, 68, 68);
private static final Color C_NODE    = new Color(71, 85, 105);
private static final Color C_NODE_START = new Color(99, 102, 241);
private static final Color C_NODE_GOAL = new Color(239, 68, 68);
private static final Color C_NODE_VISIT = new Color(245, 158, 11);
private static final Color C_NODE_PATH = new Color(16, 185, 129);
private static final Color C_EDGE     = new Color(100, 116, 139);
private static final Color C_EDGE_PATH = new Color(16, 185, 129);
private static final Color C_TEXT     = new Color(226, 232, 240);
private static final Color C_TEXT_DIM = new Color(148, 163, 184);

public PetaKota_2511532027() {
    initGraph_2027();
    initNodePositions_2027();
    visitedNodes_2027 = new ArrayList<>();
    highlightPath_2027 = new HashSet<>();
    highlightVisited_2027 = new HashSet<>();
    buildGUI_2027();
}

private void initGraph_2027() {
    graph_2027 = new LinkedHashMap<>();

    String[] nodes_2027 = {
        "Rumah", "Rumah Sakit", "Kantor Polisi", "Sekolah (SD)",
        "Sekolah (SMA)", "Masjid", "Lapangan Olahraga",

```

```

        "Pom Bensin", "Restoran Soerabi", "Swalayan",
        "Kantor Pos", "Sekolah (SMP)", "Food Court",
        "Tugu Pahlawan", "Jam Gadang", "Pasar Atas",
        "Ramayana", "Istana Bung Hatta", "Hotel",
        "KFC", "Pizza Hut"
    };

    for (String n : nodes_2027) graph_2027.put(n, new
ArrayList<>());

    String[][] edges_2027 = {
        {"Rumah", "Rumah Sakit"},
        {"Rumah", "Kantor Polisi"},
        {"Rumah", "Sekolah (SD)"},
        {"Rumah", "Sekolah (SMA)"},
        {"Kantor Polisi", "Sekolah (SD)"},
        {"Kantor Polisi", "Sekolah (SMA)"},
        {"Rumah Sakit", "Sekolah (SMA)"},
        {"Sekolah (SD)", "Masjid"},
        {"Sekolah (SMA)", "Lapangan Olahraga"},
        {"Rumah Sakit", "Lapangan Olahraga"},
        {"Lapangan Olahraga", "Pom Bensin"},
        {"Lapangan Olahraga", "Restoran Soerabi"},
        {"Restoran Soerabi", "Swalayan"},
        {"Pom Bensin", "Kantor Pos"},
        {"Kantor Pos", "Sekolah (SMP)"},
        {"Kantor Pos", "Food Court"},
        {"Sekolah (SMP)", "Food Court"},
        {"Sekolah (SMP)", "Tugu Pahlawan"},
        {"Tugu Pahlawan", "Jam Gadang"},
        {"Jam Gadang", "Pasar Atas"},
        {"Jam Gadang", "Ramayana"},
        {"Jam Gadang", "Istana Bung Hatta"},
        {"Istana Bung Hatta", "Hotel"},
        {"Hotel", "Ramayana"},
        {"Ramayana", "KFC"},
        {"KFC", "Pizza Hut"}
    };

    for (String[] e : edges_2027) {

```



```

        graph_2027.get(e[0]).add(e[1]);
        graph_2027.get(e[1]).add(e[0]);
    }
}

private void initNodePositions_2027() {
    nodePos_2027 = new LinkedHashMap<>();
    nodePos_2027.put("Rumah", new Point(80, 180));
    nodePos_2027.put("Kantor Polisi", new Point(230, 100));
    nodePos_2027.put("Sekolah (SD)", new Point(230, 240));
    nodePos_2027.put("Masjid", new Point(370, 240));
    nodePos_2027.put("Rumah Sakit", new Point(80, 310));
    nodePos_2027.put("Sekolah (SMA)", new Point(230, 360));
    nodePos_2027.put("Lapangan Olahraga", new Point(380, 360));
    nodePos_2027.put("Pom Bensin", new Point(510, 270));
    nodePos_2027.put("Restoran Soerabi", new Point(510, 430));
    nodePos_2027.put("Swalayan", new Point(640, 430));
    nodePos_2027.put("Kantor Pos", new Point(640, 270));
    nodePos_2027.put("Sekolah (SMP)", new Point(760, 180));
    nodePos_2027.put("Food Court", new Point(760, 340));
    nodePos_2027.put("Tugu Pahlawan", new Point(880, 100));
    nodePos_2027.put("Jam Gadang", new Point(980, 230));
    nodePos_2027.put("Pasar Atas", new Point(1100, 130));
    nodePos_2027.put("Ramayana", new Point(1100, 300));
    nodePos_2027.put("Istana Bung Hatta", new Point(980, 370));
    nodePos_2027.put("Hotel", new Point(1100, 430));
    nodePos_2027.put("KFC", new Point(1200, 300));
    nodePos_2027.put("Pizza Hut", new Point(1280, 400));
}

public List<String> BFS_2027(String start_2027, String goal_2027) {
    visitedNodes_2027 = new ArrayList<>();
    Map<String, String> parent_2027 = new HashMap<>();
    Queue<String> queue_2027 = new LinkedList<>();
    Set<String> seen_2027 = new LinkedHashSet<>();

    queue_2027.add(start_2027);
    seen_2027.add(start_2027);
}

```

```

        parent_2027.put(start_2027, null);

        while (!queue_2027.isEmpty()) {
            String cur_2027 = queue_2027.poll();
            visitedNodes_2027.add(cur_2027);
            if (cur_2027.equals(goal_2027)) return
buildPath_2027(parent_2027, goal_2027);
            for (String nb_2027 : graph_2027.getDefault(cur_2027,
Collections.emptyList())) {
                if (!seen_2027.contains(nb_2027)) {
                    seen_2027.add(nb_2027);
                    parent_2027.put(nb_2027, cur_2027);
                    queue_2027.add(nb_2027);
                }
            }
        }
        return null;
    }

    public List<String> DFS_2027(String start_2027, String goal_2027) {
        visitedNodes_2027 = new ArrayList<>();
        Map<String, String> parent_2027 = new HashMap<>();
        Set<String> seen_2027 = new LinkedHashSet<>();
        parent_2027.put(start_2027, null);
        boolean found_2027 = dfsRec_2027(start_2027, goal_2027,
seen_2027, parent_2027);
        return found_2027 ? buildPath_2027(parent_2027, goal_2027) :
null;
    }

    private boolean dfsRec_2027(String cur_2027, String goal_2027,
        Set<String> seen_2027,
        Map<String, String> parent_2027) {
        seen_2027.add(cur_2027);
        visitedNodes_2027.add(cur_2027);
        if (cur_2027.equals(goal_2027)) return true;
        for (String nb_2027 : graph_2027.getDefault(cur_2027,
Collections.emptyList())) {

```

```

        if (!seen_2027.contains(nb_2027)) {
            parent_2027.put(nb_2027, cur_2027);
            if (dfsRec_2027(nb_2027, goal_2027, seen_2027,
parent_2027)) return true;
        }
    }
    return false;
}

private List<String> buildPath_2027(Map<String, String> parent_2027,
String goal_2027) {
    LinkedList<String> path_2027 = new LinkedList<>();
    for (String c = goal_2027; c != null; c = parent_2027.get(c))
        path_2027.addFirst(c);
    return path_2027;
}

public void displayPath_2027(List<String> path_2027, String
algo_2027) {
    StringBuilder sb_2027 = new StringBuilder();
    sb_2027.append("[ ").append(algo_2027).append(" ]\n\n");

    if (path_2027 == null) {
        sb_2027.append("Jalur tidak ditemukan.\n\n");
        highlightPath_2027 = new HashSet<>();
    } else {
        sb_2027.append("Jalur Ditemukan :\n ")
            .append(String.join(" -> ", path_2027))
            .append("\n\n");
        highlightPath_2027 = new HashSet<>(path_2027);
    }

    sb_2027.append("Node Kunjungi :\n ")
        .append(String.join(" -> ", visitedNodes_2027))
        .append("\n\n")
        .append("Jumlah Node Dieksplorasi : ")
        .append(visitedNodes_2027.size());
}

```

```

        if (path_2027 != null)
            sb_2027.append("\nPanjang Jalur : ")
                    .append(path_2027.size() - 1).append(" langkah");

        highlightVisited_2027 = new HashSet<>(visitedNodes_2027);
        resultArea_2027.setText(sb_2027.toString());
        resultArea_2027.setCaretPosition(0);
        graphPanel_2027.repaint();
    }

    public void displayGraph_2027() {
        highlightPath_2027 = new HashSet<>();
        highlightVisited_2027 = new HashSet<>();
        resultArea_2027.setText(
            "Hasil Pencarian :\n\nJalur :\n\nNode Dikunjungi :\n\nJumlah
Node Dieksplorasi : 0");
        graphPanel_2027.repaint();
    }

    public void resetGraph_2027() {
        visitedNodes_2027 = new ArrayList<>();
        highlightPath_2027 = new HashSet<>();
        highlightVisited_2027 = new HashSet<>();
        cbStart_2027.setSelectedIndex(0);
        cbGoal_2027.setSelectedIndex(0);
        displayGraph_2027();
    }

    private void buildGUI_2027() {
        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(0, 0));
        getContentPane().setBackground(C_BG);

        JPanel header_2027 = new JPanel(new BorderLayout());
        header_2027.setBackground(C_PANEL);
        header_2027.setBorder(new EmptyBorder(14, 20, 14, 20));
        JLabel titleLbl_2027 = new JLabel(

```

```

        "PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS",
        SwingConstants.CENTER);
    titleLbl_2027.setFont(new Font("Segoe UI", Font.BOLD, 18));
    titleLbl_2027.setForeground(C_TEXT);
    header_2027.add(titleLbl_2027, BorderLayout.CENTER);
    add(header_2027, BorderLayout.NORTH);

    graphPanel_2027 = new GraphPanel_2027();
    JScrollPane graphScroll_2027 = new JScrollPane(graphPanel_2027);

graphScroll_2027.setBorder(BorderFactory.createLineBorder(C_CARD, 1));
    graphScroll_2027.getViewport().setBackground(C_BG);

    JPanel rightPanel_2027 = buildRightPanel_2027();

    JPanel centerRow_2027 = new JPanel(new BorderLayout(8, 0));
    centerRow_2027.setBackground(C_BG);
    centerRow_2027.setBorder(new EmptyBorder(6, 6, 6, 6));
    centerRow_2027.add(graphScroll_2027, BorderLayout.CENTER);
    centerRow_2027.add(rightPanel_2027, BorderLayout.EAST);
    add(centerRow_2027, BorderLayout.CENTER);

    setSize(1720, 780);
    setMinimumSize(new Dimension(1000, 600));
    setLocationRelativeTo(null);
    setVisible(true);
}

private JPanel buildRightPanel_2027() {
    JPanel p = new JPanel();
    p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));
    p.setBackground(C_BG);
    p.setBorder(new EmptyBorder(2, 6, 4, 2));
    p.setPreferredSize(new Dimension(380, 600));

    JPanel card_2027 = new JPanel(new GridBagLayout());
    card_2027.setBackground(C_PANEL);
    card_2027.setBorder(BorderFactory.createCompoundBorder(

```

```

        BorderFactory.createLineBorder(C_CARD, 1),
        new EmptyBorder(14, 14, 14, 14)));
card_2027.setMaximumSize(new Dimension(Integer.MAX_VALUE, 180));

GridBagConstraints gc = new GridBagConstraints();
gc.insets = new Insets(6, 4, 6, 4);
gc.fill = GridBagConstraints.HORIZONTAL;

String[] nodeArr_2027 = graph_2027.keySet().toArray(new
String[0]);

gc.gridx = 0; gc.gridy = 0; gc.weightx = 0;
card_2027.add(makeLabel_2027("Lokasi Awal :"), gc);
gc.gridx = 1; gc.weightx = 1;
cbStart_2027 = makeCombo_2027(nodeArr_2027);
card_2027.add(cbStart_2027, gc);

gc.gridx = 0; gc.gridy = 1; gc.weightx = 0;
card_2027.add(makeLabel_2027("Lokasi Tujuan :"), gc);
gc.gridx = 1; gc.weightx = 1;
cbGoal_2027 = makeCombo_2027(nodeArr_2027);
cbGoal_2027.setSelectedIndex(Math.min(5, nodeArr_2027.length -
1));
card_2027.add(cbGoal_2027, gc);

JPanel btnRow_2027 = new JPanel(new GridLayout(1, 3, 8, 0));
btnRow_2027.setBackground(C_PANEL);
btnRow_2027.setMaximumSize(new Dimension(Integer.MAX_VALUE,
38));

JButton btnBFS_2027 = makeButton_2027("[ BFS ]", C_GREEN);
btnBFS_2027.addActionListener(e -> {
    String s = (String) cbStart_2027.getSelectedItem();
    String g = (String) cbGoal_2027.getSelectedItem();
    if (s.equals(g)) {
        resultArea_2027.setText("Lokasi awal dan tujuan tidak
boleh sama!");
    }
    return;
});

```

```

        }
        displayPath_2027(BFS_2027(s, g), "BFS (Breadth First
Search)");
    });

    JButton btnDFS_2027 = makeButton_2027("[ DFS ]", C_YELLOW);
    btnDFS_2027.addActionListener(e -> {
        String s = (String) cbStart_2027.getSelectedItemAt();
        String g = (String) cbGoal_2027.getSelectedItemAt();
        if (s.equals(g)) {
            resultArea_2027.setText("Lokasi awal dan tujuan tidak
boleh sama!");
            return;
        }
        displayPath_2027(DFS_2027(s, g), "DFS (Depth First
Search)");
    });

    JButton btnReset_2027 = makeButton_2027("[ RESET ]", C_RED);
    btnReset_2027.addActionListener(e -> resetGraph_2027());

    btnRow_2027.add(btnBFS_2027);
    btnRow_2027.add(btnDFS_2027);
    btnRow_2027.add(btnReset_2027);

    gc.gridx = 0; gc.gridy = 2; gc.gridwidth = 2; gc.weightx = 1;
    gc.insets = new Insets(14, 0, 4, 0);
    card_2027.add(btnRow_2027, gc);

    p.add(card_2027);
    p.add(Box.createVerticalStrut(10));

    JLabel resultTitle_2027 = new JLabel("Hasil Pencarian");
    resultTitle_2027.setFont(new Font("Segoe UI", Font.BOLD, 13));
    resultTitle_2027.setForeground(C_TEXT);
    resultTitle_2027.setAlignmentX(Component.LEFT_ALIGNMENT);
    p.add(resultTitle_2027);
    p.add(Box.createVerticalStrut(4));

```

```

        resultArea_2027 = new JTextArea();
        resultArea_2027.setFont(new Font("Consolas", Font.PLAIN, 11));
        resultArea_2027.setForeground(C_TEXT);
        resultArea_2027.setBackground(C_CARD);
        resultArea_2027.setCaretColor(C_TEXT);
        resultArea_2027.setEditable(false);
        resultArea_2027.setLineWrap(true);
        resultArea_2027.setWrapStyleWord(true);
        resultArea_2027.setBorder(new EmptyBorder(8, 10, 8, 10));
        resultArea_2027.setText(
            "Hasil Pencarian :\n\nJalur :\n\nNode Dikunjungi :\n\nJumlah
Node Dieksplorasi : 0");

        JScrollPane rScroll_2027 = new JScrollPane(resultArea_2027);
        rScroll_2027.setBorder(BorderFactory.createLineBorder(C_ACCENT,
1));

        rScroll_2027.setAlignmentX(Component.LEFT_ALIGNMENT);
        rScroll_2027.setPreferredSize(new Dimension(370, 260));
        rScroll_2027.setMaximumSize(new Dimension(Integer.MAX_VALUE,
300));

        p.add(rScroll_2027);
        p.add(Box.createVerticalStrut(10));

        p.add(buildLegend_2027());

        return p;
    }

    private JPanel buildLegend_2027() {
        JPanel p = new JPanel();
        p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));
        p.setBackground(C_BG);

        JLabel title_2027 = new JLabel("Keterangan Warna Node :");
        title_2027.setFont(new Font("Segoe UI", Font.BOLD, 12));
        title_2027.setForeground(C_TEXT_DIM);
        p.add(title_2027);
    }

```



```

        p.add(Box.createVerticalStrut(4));

        addLegRow_2027(p, C_NODE_START, "Node Awal (Start)");
        addLegRow_2027(p, C_NODE_GOAL, "Node Tujuan (Goal)");
        addLegRow_2027(p, C_NODE_PATH, "Node dalam Jalur");
        addLegRow_2027(p, C_NODE_VISIT, "Node Dikunjungi");
        addLegRow_2027(p, C_NODE, "Node Biasa");
        return p;
    }

    private void addLegRow_2027(JPanel p, Color c, String txt) {
        JPanel row = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 1));
        row.setBackground(C_BG);
        JLabel dot = new JLabel("●");
        dot.setFont(new Font("Segoe UI", Font.PLAIN, 16));
        dot.setForeground(c);
        JLabel lbl = new JLabel(txt);
        lbl.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        lbl.setForeground(C_TEXT_DIM);
        row.add(dot); row.add(lbl);
        p.add(row);
    }

    private JLabel makeLabel_2027(String text) {
        JLabel l = new JLabel(text);
        l.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        l.setForeground(C_TEXT);
        return l;
    }

    private JComboBox<String> makeCombo_2027(String[] items) {
        JComboBox<String> cb = new JComboBox<>(items);
        cb.setFont(new Font("Segoe UI", Font.PLAIN, 12));

        cb.setRenderer(new BasicComboBoxRenderer() {
            public Component getListCellRendererComponent(
                JList list, Object value, int index,
                boolean isSelected, boolean cellHasFocus) {

```

```

        JLabel lbl = (JLabel)
super.getListCellRendererComponent(
        list, value, index, isSelected, cellHasFocus);
        lbl.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        if (isSelected) {
            lbl.setBackground(C_ACCENT);
            lbl.setForeground(Color.WHITE);
        } else {
            lbl.setBackground(C_CARD);
            lbl.setForeground(C_TEXT);
        }
        lbl.setBorder(new EmptyBorder(4, 8, 4, 8));
        return lbl;
    }
});

cb.setUI(new javax.swing.plaf.basic.BasicComboBoxUI() {
    @Override
    protected JButton createArrowButton() {
        JButton btn = new JButton("▼");
        btn.setFont(new Font("Segoe UI", Font.PLAIN, 9));
        btn.setForeground(C_TEXT);
        btn.setBackground(C_CARD);
        btn.setBorder(BorderFactory.createEmptyBorder(0, 4, 0,
4));

        btn.setFocusPainted(false);
        btn.setContentAreaFilled(true);
        return btn;
    }

    @Override
    public void paintCurrentValue(Graphics g, Rectangle bounds,
boolean hasFocus) {

        g.setColor(C_CARD);
        g.fillRect(bounds.x, bounds.y, bounds.width,
bounds.height);

```

```

        String val = (String) comboBox.getSelectedItemAt();
        if (val != null) {
            Graphics2D g2 = (Graphics2D) g;

g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
            g2.setFont(new Font("Segoe UI", Font.PLAIN, 12));
            g2.setColor(C_TEXT);
            FontMetrics fm = g2.getFontMetrics();
            int y = bounds.y + (bounds.height + fm.getAscent() -
fm.getDescent()) / 2;
            g2.drawString(val, bounds.x + 6, y);
        }
    }

    @Override
    public void paintCurrentValueBackground(Graphics g,
Rectangle bounds, boolean hasFocus) {
        g.setColor(C_CARD);
        g.fillRect(bounds.x, bounds.y, bounds.width,
bounds.height);
    }
});

    cb.setBackground(C_CARD);
    cb.setForeground(C_TEXT);
    cb.setBorder(BorderFactory.createLineBorder(C_CARD, 1));
    cb.setMaximumRowCount(12);
    cb.setPreferredSize(new Dimension(220, 30));
    return cb;
}

private JButton makeButton_2027(String text, Color bg) {
    JButton btn = new JButton(text) {
        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();

```

```

        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                             RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setColor(getModel().isPressed() ? bg.darker() :
                     getModel().isRollover() ? bg.brighter() :
bg);

        g2.fillRoundRect(0, 0, getWidth(), getHeight(), 10, 10);
        g2.setColor(Color.WHITE);
        g2.setFont(getFont());
        FontMetrics fm = g2.getFontMetrics();
        g2.drawString(getText(),
                       (getWidth() - fm.stringWidth(getText())) / 2,
                       (getHeight() + fm.getAscent() - fm.getDescent()) /
2);

        g2.dispose();
    }
};

btn.setFont(new Font("Segoe UI", Font.BOLD, 12));
btn.setPreferredSize(new Dimension(100, 34));
btn.setMinimumSize(new Dimension(80, 34));
btn.setContentAreaFilled(false);
btn.setBorderPainted(false);
btn.setFocusPainted(false);
btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
return btn;
}

class GraphPanel_2027 extends JPanel {
    /**
     *
     */
    private static final long serialVersionUID = 1L;
    private static final int R = 26;

    GraphPanel_2027() {
        setBackground(C_BG);
        setPreferredSize(new Dimension(1310, 530));
    }
}

```

```

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
    g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
        RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
    drawEdges_2027(g2);
    drawNodes_2027(g2);
}

private void drawEdges_2027(Graphics2D g2) {
    Set<String> drawn_2027 = new HashSet<>();
    for (Map.Entry<String, List<String>> entry :
graph_2027.entrySet()) {
        String from_2027 = entry.getKey();
        Point pf_2027 = nodePos_2027.get(from_2027);
        if (pf_2027 == null) continue;

        for (String to_2027 : entry.getValue()) {
            String key_2027 = from_2027.compareTo(to_2027) < 0
                ? from_2027 + "||" + to_2027
                : to_2027 + "||" + from_2027;
            if (drawn_2027.contains(key_2027)) continue;
            drawn_2027.add(key_2027);

            Point pt_2027 = nodePos_2027.get(to_2027);
            if (pt_2027 == null) continue;

            boolean onPath_2027 =
highlightPath_2027.contains(from_2027)
                &&
highlightPath_2027.contains(to_2027);
            if (onPath_2027) {
                g2.setStroke(new BasicStroke(3.5f,
                    BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));

```

```

        g2.setColor(C_EDGE_PATH);
    } else {
        g2.setStroke(new BasicStroke(1.8f,
            BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
        g2.setColor(C_EDGE);
    }
    g2.drawLine(pf_2027.x, pf_2027.y, pt_2027.x,
pt_2027.y);
    }
}

private void drawNodes_2027(Graphics2D g2) {
    String start_2027 = (String) cbStart_2027.getSelectedItem();
    String goal_2027 = (String) cbGoal_2027.getSelectedItem();

    for (Map.Entry<String, Point> entry :
nodePos_2027.entrySet()) {
        String node_2027 = entry.getKey();
        Point p_2027 = entry.getValue();

        Color fill_2027;
        if (node_2027.equals(start_2027))
fill_2027 = C_NODE_START;
        else if (node_2027.equals(goal_2027))
fill_2027 = C_NODE_GOAL;
        else if (highlightPath_2027.contains(node_2027))
fill_2027 = C_NODE_PATH;
        else if (highlightVisited_2027.contains(node_2027))
fill_2027 = C_NODE_VISIT;
        else
fill_2027 = C_NODE;

        g2.setColor(new Color(0, 0, 0, 70));
        g2.fillOval(p_2027.x - R + 3, p_2027.y - R + 3, R * 2, R
* 2);

```

```

        g2.setColor(fill_2027);
        g2.fillOval(p_2027.x - R, p_2027.y - R, R * 2, R * 2);

        g2.setStroke(new BasicStroke(2f));
        g2.setColor(fill_2027.brighter());
        g2.drawOval(p_2027.x - R, p_2027.y - R, R * 2, R * 2);

        g2.setColor(Color.WHITE);
        drawLabel_2027(g2, node_2027, p_2027);
    }
}

private void drawLabel_2027(Graphics2D g2, String name_2027,
Point p_2027) {
    Font font_2027 = new Font("Segoe UI", Font.BOLD, 8);
    g2.setFont(font_2027);
    FontMetrics fm_2027 = g2.getFontMetrics();

    String[] words_2027 = name_2027.split(" ");
    List<String> lines_2027 = new ArrayList<>();

    if (name_2027.length() <= 8 || words_2027.length == 1) {
        lines_2027.add(name_2027);
    } else {
        int mid_2027 = (words_2027.length + 1) / 2;
        StringBuilder b1 = new StringBuilder(), b2 = new
StringBuilder();
        for (int i = 0; i < words_2027.length; i++) {
            if (i < mid_2027) { if (b1.length() > 0) b1.append('
'); b1.append(words_2027[i]); }
            else { if (b2.length() > 0) b2.append('
'); b2.append(words_2027[i]); }
        }
        lines_2027.add(b1.toString());
        if (b2.length() > 0) lines_2027.add(b2.toString());
    }

    int lh_2027 = fm_2027.getHeight();

```

```

        int y_2027 = p_2027.y - lh_2027 * lines_2027.size() / 2 +
fm_2027.getAscent() - 1;
        for (String ln_2027 : lines_2027) {
            g2.drawString(ln_2027, p_2027.x -
fm_2027.stringWidth(ln_2027) / 2, y_2027);
            y_2027 += lh_2027;
        }
    }
}

public static void main(String[] args) {
    try {
        for (UIManager.LookAndFeelInfo info :
UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (Exception ignored) {}

    SwingUtilities.invokeLater(PetaKota_2511532027::new);
}
}

```

Kode program tersebut digunakan untuk membangun sebuah aplikasi visualisasi pencarian jalur pada graf menggunakan algoritma BFS dan DFS berbasis Java Swing. Program merepresentasikan sebuah peta kota yang terdiri dari berbagai lokasi, seperti rumah, rumah sakit, sekolah, dan tempat umum lainnya. Selain melakukan pencarian jalur, aplikasi juga mampu menampilkan graf secara visual sehingga pengguna dapat melihat hubungan antar lokasi dan hasil pencarian secara langsung.

Kode program tersebut digunakan untuk menyimpan struktur graf menggunakan variabel `graph_2027` yang bertipe `Map<String, List<String>>`. Struktur ini berfungsi sebagai *adjacency list*. Method `initGraph_2027()` bertugas menginisialisasi seluruh simpul dan hubungan antar lokasi. Hubungan tersebut

bersifat dua arah, sehingga setiap lokasi dapat diakses dari lokasi lain yang terhubung secara langsung.

Kode program tersebut juga digunakan untuk menentukan posisi setiap simpul pada tampilan graf melalui method `initNodePositions_2027()`. Method ini menyimpan koordinat masing-masing lokasi menggunakan objek `Point`, sehingga setiap node dapat digambar pada posisi tertentu di dalam panel graf. Dengan adanya koordinat ini, tampilan graf menjadi lebih terstruktur dan menyerupai peta kota yang sebenarnya.

Kode program tersebut digunakan untuk mengimplementasikan algoritma BFS melalui method `BFS_2027()`. Algoritma BFS bekerja dengan menggunakan struktur data *Queue* untuk menelusuri simpul berdasarkan tingkat kedekatannya dari simpul awal. Selama proses pencarian, setiap node yang dikunjungi disimpan ke dalam daftar `visitedNodes_2027`, sedangkan hubungan antar node disimpan dalam variabel `parent_2027` untuk membangun kembali jalur yang ditemukan. Apabila node tujuan berhasil ditemukan, program akan mengembalikan jalur terpendek dari lokasi awal menuju lokasi tujuan.

Selain itu, kode program tersebut digunakan untuk mengimplementasikan algoritma DFS melalui method `DFS_2027()` dan `dfsRec_2027()`. DFS melakukan penelusuran secara mendalam hingga mencapai node terdalam sebelum kembali ke node sebelumnya. Implementasi DFS dilakukan secara rekursif dan menggunakan struktur data *Set* untuk menyimpan node yang telah dikunjungi agar tidak terjadi perulangan penelusuran. Jika node tujuan ditemukan, program akan membangun jalur menggunakan method `buildPath_2027()`.

Kode program tersebut juga digunakan untuk membangun jalur hasil pencarian menggunakan method `buildPath_2027()`. Method ini memanfaatkan informasi *parent* dari setiap node untuk menyusun urutan lokasi dari titik awal hingga titik tujuan. Jalur yang terbentuk kemudian ditampilkan kepada pengguna sebagai hasil pencarian BFS maupun DFS.

Kode program tersebut digunakan untuk menampilkan hasil pencarian melalui method `displayPah_2027()`. Method ini menampilkan informasi berupa jalur yang ditemukan daftar node yang dikunjungi selama proses pencarian, jumlah node yang dieksplorasi, serta panjang jalur yang berhasil ditemukan. Selain itu, node-

node yang termasuk dalam jalur dan node-node yang telah dikunjungi akan diberikan warna khusus pada tampilan graf sehingga pengguna dapat membedakan hasil pencarian dengan lebih mudah.

Kode program tersebut juga digunakan untuk membangun GUI menggunakan Java Swing. Method `buildGUI_2027()` berfungsi menyusun seluruh komponen tampilan, seperti judul aplikasi, area visualisasi graf, panel kontrol, serta panel hasil pencarian. Pada panel kontrol, pengguna dapat memilih lokasi awal dan lokasi tujuan melalui komponen `JComboBox`, kemudian menjalankan algoritma BFS atau DFS menggunakan tombol yang tersedia.

Selain itu, kode program tersebut digunakan untuk memberikan tampilan yang lebih menarik dan interaktif. Method `makeCombo_2027()`, `makeButton_2027()`, dan `buildLegend_2027()` berfungsi mengatur desain komponen GUI, seperti warna, font, ikon, serta legenda warna yang menjelaskan arti setiap node pada graf. Warna tersebut digunakan untuk menandai node awal, node tujuan, node yang dikunjungi, dan node yang termasuk dalam jalur hasil pencarian.

Kode progra tersebut digunakan untuk menggambar graf secara visual melalui kelas `GraphPanel_2027` yang merupakan turunan dari `JPanel`. Method `drawEdges_2027()` digunakan untuk menggambar garis penghubung antar node, sedangkan method `drawNodes_2027()` digunakan untuk menggambar node berbentuk lingkaran yang mewakili setiap lokasi. Apabila suatu node termasuk dalam jalur hasil pencarian atau telah dikunjungi oleh algoritma, warna node dan edge akan berubah sesuai kategori yang telah ditentukan sehingga proses pencarian dapat diamati secara visual.

Terakhir, kode program tersebut digunakan untuk menjalankan aplikasi melalui method `main()`. Method ini mengatur tampilan agar antarmuka terlihat lebih modern, kemudian membuat objek `PetaKota_2511532027` menggunakan `SwingUtilities.invokeLater()`.

Hasil / Output :



BAB 3

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Binary Tree* dan *Graph* merupakan struktur data non-linear yang digunakan untuk merepresentasikan hubungan antar data secara lebih kompleks dibandingkan struktur data linear. *Binary Tree* memiliki struktur hierarkis yang terdiri atas *root*, *parent*, *child*, dan *leafnode*, sehingga cocok digunakan dalam proses pencarian dan pengelolaan data yang terorganisir. Sementara itu, *Graph* terdiri atas *vertex* dan *edge* yang mampu menggambarkan berbagai hubungan antar objek secara fleksibel. Pemahaman mengenai kedua struktur data tersebut sangat penting karena menjadi dasar dalam pengembangan berbagai algoritma dan aplikasi komputer yang membutuhkan pengelolaan secara efisien.

Dalam mempelajari *Binary Tree* dan *Graph*, disarankan untuk lebih banyak berlatih mengimplementasikan berbagai operasi dan algoritma yang berkaitan dengan kedua struktur data tersebut agar pemahaman konsep menjadi lebih mendalam. Selain itu, perlu dilakukan eksplorasi terhadap variasi struktur dan metode penelusuran yang lebih kompleks, sehingga kemampuan dalam menganalisis serta menyelesaikan permasalahan menggunakan struktur data non-linear dapat terus berkembang.

DAFTAR PUSTAKA

[1] <https://www.geeksforgeeks.org/dsa/binary-tree-data-structure/> [Diakses 09 Juni 2026]

[2] <https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/>
[Diakses 09 Juni 2026]

[3] <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
[Diakses 09 Juni 2026]

[4] <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
[Diakses 09 Juni 2026]

LAMPIRAN

Gambar 2.1	15
Gambar 2.2	16
Gambar 2.3	17
Gambar 2.4	18
Gambar 2.5	40
Gambar 2.6	40